POLITÉCNICO DE LEIRIA | ESCOLA SUPERIOR DE TECNOLOGIA E GESTÃO

Advanced Game Programming Topics

Computer Graphics

Project 2 Report

Undergraduate in Games and Multimedia

2025/2026

Student

2230775 | Alexander Chicharo

Introduction

This project implements a small 2D game engine in C++ (BoberEngine) and a Xenon 2000 clone built on top of it. The engine handles the update, rendering, sprite rendering, animation, input and physics using Box2D

The project uses two external libraries , SDL3 and Box2D

Implemented Features

- Separated solution for the engine and game.

- Actor/level system, where the level owns the actors and calls their update and render

- Importing .bmp images as spritesheets with purple (magenta) color key for alpha transparency and animation clips

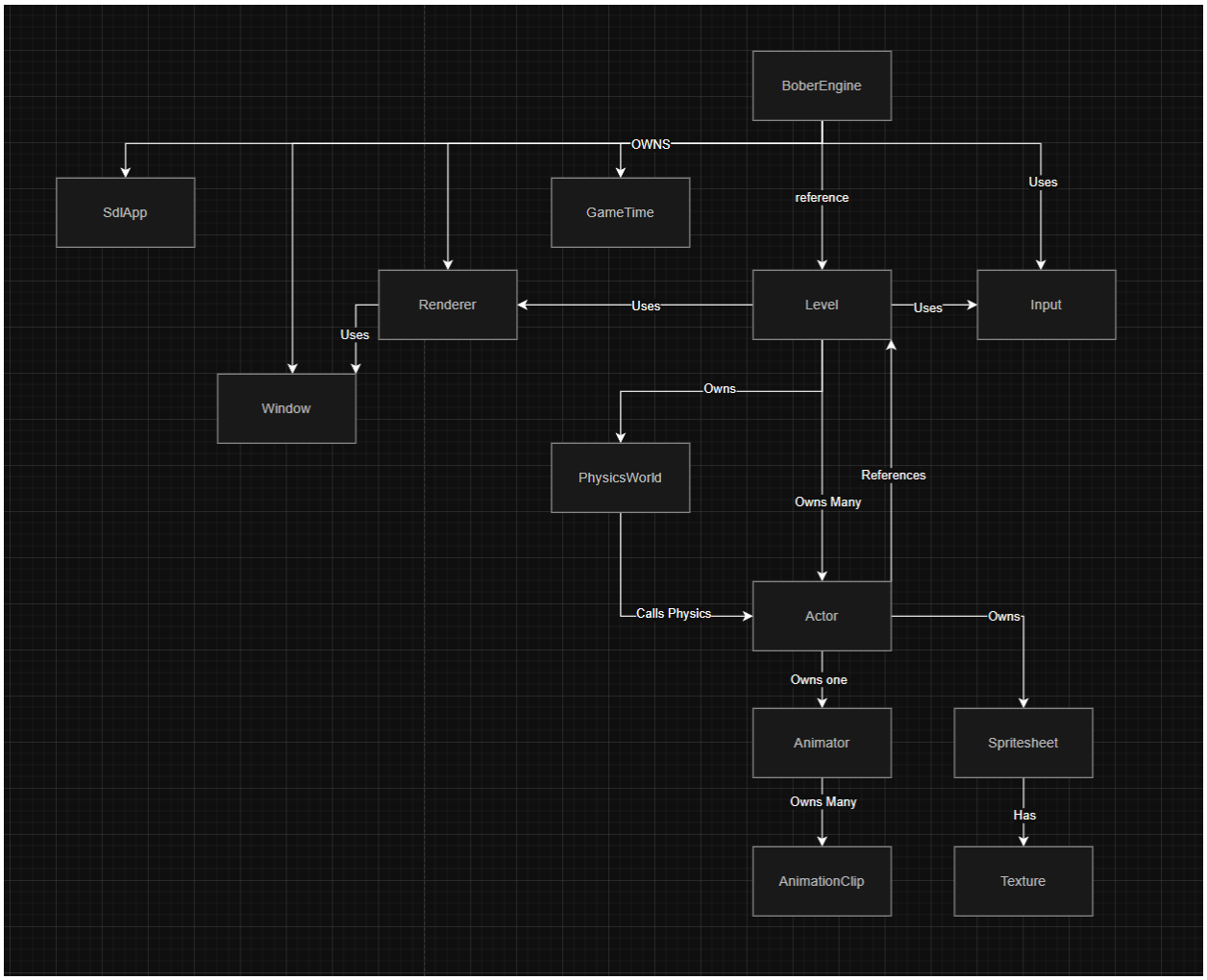
- Box2D for physics , colliders and collision callbacks for use in actors.

- In game side for the parallax implemented Wave function collapse (WFC) for generating the side walls procedurally.

- Gameplay: player ship, missiles, enemies (Loner/Rusher), enemy bullets, explosions, Paralax.

Class Diagram

The following diagram was made to summarize the main engine classes and how they connect.



Engine Description (BoberEngine)

- **Main loop (BoberEngine)**

  BoberEngine owns the SDL application objects, handles the event loop, and runs the main loop. Every frame it processes input/events, updates the current Level, then renders it.

- **SDL/Opengl GLSL wrappers**

  SDL Usage is wrapped in small classes: SdlApp initializes SDL, Window creates the window, and Renderer wraps some basic functions of sdl. Texture.cpp/.h is what mostly renders stuff to the screen and is what's called in the end of the rendering pipeline following: BoberEngine calls Level, Level calls Actor, Actor calls either spritesheet or Texture, in case of calling spritesheet, spritesheet calls Texture.

  Texture.cpp now uses OpenGL+ GLSL to render the sprites into squares instead of rendering directly the pixels to screen like sdl does.

  Added a function to rotate the sprite rendering as i didnt have it in the SDL version of the project, because the game runs horizontally i had to rotate the actors sprites.

- **Level / Actor**

  Level owns a list of Actor objects and also Updates and Renders them. Actors handle their own logic (update) and can also handle their own rendering by overwriting it but that doesn't happen in Xenon. Render order is controlled by an integer render layer. Level sorts actors based on the layer each frame before drawing.

  To avoid crashes when spawning actors during iteration, Level uses staged spawning: actors created during update/collision are placed into a pending list and committed after the loop. This commit of actors appears many times in the Level.cpp code.

- **Input**

  Input supports keyboard and controller, only some buttons have been coded in, mostly only the necessary for xenon. SDL events are converted into a stable input state (keys/buttons/axes), then gameplay code gets the Input each frame.

- **PhysicsWorld and collisions**

  PhysicsWorld is a wrapper around a Box2D world. Actors can create a body, add colliders, and set velocities using helper functions. After stepping physics with a fixed timestep, contact events are forwarded to actors through collision callbacks.

Xenon Game

- **XenonLevel**

  XenonLevel sets the background, creates world bounds, spawns the player ship, and randomly spawns enemies using timers. It also updates the SideWallsLayer. Loners are set to spawn only 6 at a time max.

- **SpaceShip**

  The player ship moves with keyboard (WASD/Arrows) and controller (left joystick). Banking (the inclination animation) is done by selecting a frame based on horizontal input, this can be seen in the controller that the slower the player moves sideways the less banking the space ship has. Shooting spawns PlayerMissile actors. When hit, the ship becomes invulnerable for a short time and blinks. The blinking “Animation” is on the engine itself.

- **Enemies, bullets, explosions**

  Loners move vertically and fire bullets right to left of the screen. Rushers move right to left of the screen. Enemy bullets and missiles use colliders and collision callbacks to detect hits. If the player missile detects a hit it’s destroyed and spawns an explosion in its place.

Explosions use a non looping animation, so the animation only plays once, and when the animation stops it gets destroyed.

Computer Graphics

replaces the SDL renderer used in Project 1 with an OpenGL 2D rendering. SDL3 is still used for window creation and event handling, but all sprite drawing is done with OpenGL + GLSL.

In the Texture.cpp swapped the sdl rendering code with code from 1st project computer graphics and adapted it for 2D, kept some of the logic from the sdl but updated to open GL so its not even close to beeing 1:1 to the computer graphics project 1, its closer to the project 1 of AGPT

Text is rendered as actors in an array by the HUD in the Xenon project, UI is also just actors, all rendered in a realy high layer to be above the game (500)

Technical Decisions / Algorithms

- **Fixed-step physics**

  Physics is stepped using a fixed timestep accumulator (1/60s). The engine clamps the number of steps per frame, which prevents instability if the frame time spikes.

- **Staged spawning**

  Actors can spawn other actors during update/collision without breaking the current iteration. This avoids crashes because of the vector with the actors variating while it is running the actor loop.

- **Side walls (Wave Function Collapse)**

  Wave function collapse is an algorithm that based on the tag of an edge of a tile will try to find at random a tile that has a compatible tag, this checks all 4 sides for compatibility against the already placed tiles, this can be used as a simple image

generation by by generating an image with some tiles and only connecting tiles where the edges have the same pixels, or for world generation, because it can only work on tiles usually for generating cities, by separating where houses or roads connect.

For this project a small set of stone wall tiles were defined with edge connectivity of just stone or air. The walls are separated in 3 columns, and default edge values were added to the extremes, making the connection with the screen borders be stone and with the inner part of the game be Air, the WFC algorithm fills the 3-column grid by selecting tiles that satisfy adjacency rules. Actors are spawned for each tile, they are rendered using the simple image, so they don't animate.

I chose the wave function collapse to create the walls because I have worked with it allot before in older projects

Achieved Objectives

- Working SDL3/OpenGL engine
- Xenon clone gameplay loop
- Input with keyboard and controller
- Sprite sheets and animations
- Box2D physics and collisions
- Procedural side walls using WFC
- Actor/Level class update and render loop

- Animations
- Horizontal Scrolling conversion
- HUD/UI
- TextRendering

Not achieved

- Dedicated Pawn class

Build / Run Instructions

Engine Library Build

1. Open the Engine solution in Visual Studio.
2. Select x64 / Debug or Release.
3. Build the solution (an error will pop out in the end trying to execute the .lib) ignore the error, if in the output on the bottom it has 0 errors then its good
4. Go into the build folder/x64/Debug
5. Copy the folder contents and paste them into EngineLib/lib
6. Copy all files from GameEngine/Headers into EngineLib/Include/BoberEngine

Add it to the game and build the game

7. Copy the EngineLib folder and paste into the game folder
8. Open the Game solution in Visual Studio.
9. Select x64 / Debug or Release.
10. Copy the graphics folder to the build folder
11. Run Xenon.exe to check if everything works

References

This project code was a mashup from basically both project 1 from both classes, so i dont have many references.

ChatGpt was still used to help me with separating the project into two.